Celestia

A Beginner-Friendly Programming Language

Faviana Troshani
Sofio Kvavadze
Vrutti Patel
Sabrina Abdukadirova
CMPSC 470 · Spring 2026

Why We Created Celestia?

- Learning programming for the first time can be difficult
- Languages like Python or C++ can feel overwhelming
- There are many rules, syntax, and confusing error messages
- Beginners struggle to focus on logic instead of syntax

What is Celestia?

Celestia is a simple programming language designed specifically for beginners. Its main goal is to teach core programming concepts such as variables, assignments, and type checking in a clear and easy way. It removes unnecessary complexity so that users can focus on understanding how programs work instead of worrying about complicated syntax.

Who is it for?

- Students with no programming experience
- Middle and high school learners
- Beginners who find Python or C++ overwhelming
- Anyone looking for a simple introduction to coding

Key Features of Celestia

Celestia has three key features that makes it simple:

- **3 Simple Types**: num, String, and Boolean. No confusing integer vs float distinction. One number type handles everything.
- **Static Type Safety**: once a variable is declared as num, you can only store numbers in it. Type mismatches are caught immediately with a clear error message.
- **Readable Syntax**: variables look like plain English. num age = 17, String name = “Alice”, Boolean active = true. Comments use // and /* */.

```
Num age = 17   // age is 17
Num gpa = 3.87
String name = "Alice"
/* This is
a comment*/
```

Data Types

Celestia supports three simple data types used to store values:

1. **num**
   - Used for all numeric values (integers and decimals)
   - Example:

```
num age = 17
num gpa = 3.87
```

2. **String**
   - Used to store text inside double quotes

```
String name = "Alice"
```

3. **Boolean**
   - Stores true or false values

```
Boolean active = true
Boolean passed = false
```

Variable Declaration

Variable declaration is one of the most important parts of Celestia because it creates the foundation for how the program stores and manages information. Before a value can be updated, printed, or checked, the variable must first be declared with a clear type and starting value.

```
num score = 80
String name = "Bob"
Boolean enrolled = true
```

Importance in Celestia

- Helps the program know what kind of data is being stored
- Prevents undeclared variables from being used by mistake
- Supports type safety by making sure values match their declared type
- Allows the symbol table to track each variable correctly
- Makes beginner code easier to read and understand

Variable declaration makes Celestia more reliable because it gives every variable a clear identity before the program continues.

Assignment

- After a variable is declared, its value can be updated at any time. The new value must match the variable's original type or the compiler will reject it.

```
score = 90
name = "Alice"
enrolled = false
```

- When an assignment occurs, Celestia looks up the variable in the symbol table and updates its stored value while preserving its original type.

Output

- Assignment allows a program to update the value of a variable after it has been declared. This is essential for creating dynamic programs where data can change during execution.

```
print score
print "Hello Celestia!"
print name
```

Importance in Celestia

- Displays values stored in variables
- Provides feedback to the user
- Helps verify that the program is working correctly
- Supports debugging by showing intermediate results

Connection to the System

The print operation retrieves values from the symbol table and outputs them to the console in a readable format.

How It Works — The Symbol Table

Declare

num age = 17 is tokenized by the Lexer, parsed into an AST node, validated by the Semantic Analyzer, and stored in the symbol table with name, type, and value.

Update

age = 18 looks up the existing entry and overwrites the value. The type must still match or the update is rejected.

Type Check

Assigning x = “hello” to a num variable gives a clear TypeError. Assigning to an undeclared variable also gives a descriptive error.

Code Generation

The Code Generator translates the validated AST into executable Python code, which is saved as a compiled file and optionally run directly.

Symbol Table

The symbol table is backed by a Python dict (hash map) keyed by variable name. Each entry stores the name, type, and current value.

Compiler Driver

The compile_source() function orchestrates all stages: Lexer → Parser → Semantic Analyzer → Code Generator. The main() function handles file input and CLI flags (--run, --show, --out).

Name	Type	Value
age	num	17
name	String	"Alice"
enrolled	Boolean	true

- Celestia uses a multi-stage compilation process that is similar to real-world compilers. The code passes through several stages.
- Each stage checks for errors and transforms the program into a more structured form before moving forward. This ensures that the final program is correct and executable.

CELESTIA COMPILER PIPELINE

How Celestia Processes Your Code

- Each stage transforms the program
- Errors are caught early
- Only valid programs execute

1 SOURCE CODE

You write code in a .cel file.

Example:

```
num x = 10
x = x + 5
print x
```

2 LEXER (Lexical Analysis)

Breaks the source code into tokens. Removes spaces, comments, and grouping.

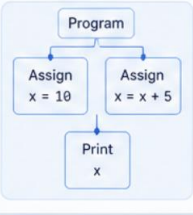
Example Tokens:

```
[KEYWORD] num
[IDENTIFIER] x
[OPERATOR] =
[NUMBER] 10
[IDENTIFIER] x
[OPERATOR] +
[NUMBER] 5
[KEYWORD] print
...
```

3 PARSER (Syntax Analysis)

Checks syntax rules and builds an Abstract Syntax Tree (AST).

AST Example:



4 SEMANTIC ANALYZER (Semantic Analysis)

Checks logical correctness. Verifies types, declarations, and valid operations using the Symbol Table.

Symbol Table Example:

Name	Type	Value
x	num	15

Checks:

- Declared before use
- Type consistency
- Valid operations

5 CODE GENERATOR (Code Generation)

Converts the AST into equivalent Python code.

Generated Python Code:

```
x = 10
x = x + 5
print(x)
```

6 OUTPUT (Execution)

The Python code is executed and produces output.

Program Output:

```
15
```

KEY BENEFITS

Fast & Efficient
Simple pipeline ensures quick processing.

Early Error Detection
Errors caught at each stage prevent invalid programs.

Beginner Friendly
Helps users understand how programs work internally.

Reliable Execution
Only valid programs reach the output stage.

Future Improvements

- Add support for functions and user-defined procedures
- Add arrays and structured data types
- Add user input support so programs can accept runtime values
- Add a GUI-based symbol table visualizer

Conclusion

We hope for Celestia to spread far, encouraging the newest generation of programmers to start learning, especially those who are young. Through the simplistic writing style and beginner friendly concept, Celestia will help branch new coders to different programs through a stepping stool that allows them to grow.